claude-windows / 166d7059-d8ac-4fdd-a8c0-072cdaa7eccd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\166d7059-d8ac-4fdd-a8c0-072cdaa7eccd\subagents\workflows\wf_66c34d9d-057\agent-a5122a38258aca720.jsonl`
- SHA-256: `ac3b6c3187513deeee0a5535f1e855e5d2be662eb561ad18fdc5fb0ad21359e5`
- Source modified: `2026-06-18T05:34:03+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_66c34d9d-057`
- session_id: `166d7059-d8ac-4fdd-a8c0-072cdaa7eccd`

Transcript

1. user (2026-06-18T05:31:43.304Z)

```
WORKTREE (cd here; code is origin/master f5f339a): C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13
Run python as:  cd "C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13" && PYTHONPATH=. python <your_script>
GOLDEN/manifest/features dir (ABSOLUTE ? pass this, the worktree output/ is empty):
C:/Users/avidu/Projects/badminton-highlight-indexer/output

REPRODUCE RECIPE (use the public API; write your own short script to scratch/lens_<name>.py):
  import os
  from dataclasses import replace
  from backend.eval import rally_seg_eval as rse, eval_stats
  from backend.eval.windowing import SERVED
  from backend.eval.tolerance_metrics import over_seg_guard
  OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"
  golden = [g for g in rse.load_golden(os.path.join(OUT,"human_lovo_manifest.json"), OUT)
            if g["gts"] and os.path.exists(g["trajectory"])]   # expect 13
  baseline  = SERVED                                          # all windowing knobs OFF
  cap12     = replace(SERVED, max_window_duration=12.0)
  cap6      = replace(SERVED, max_window_duration=6.0)
  milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                      inactivity_rally_thresh=0.20, inactivity_min_density=0.30)
  rows = lambda p: {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}  # each row: tol_f1,
      # f1, merge_rate, split_rate, merge_count, split_count, num_pred, num_gt, name,
dend_abs_median
  # paired sign-test on a metric: eval_stats.paired_delta_ci([base[n][k] for n in
names],[cand[n][k] for n in names])
  #   returns {mean_delta, ci_low, ci_high, ci_excludes_zero,
sign_test:{wins,losses,ties,p_value}}
  # over-seg guard: over_seg_guard(list(base_rows.values()), list(cand_rows.values()))
  #   returns {passes, strict_passes, base_net, cand_net, net_delta, no_merge_rate_regress,
merge_rate_regress:[...]}
  # NOTE: rse.evaluate may print "R5 blob-fallback: forced ..." lines for the blob clip ? that
is the
  #   R5 logger and is HARMLESS/irrelevant here (the milestone does NOT enable blob_fallback).
Ignore it.

THE CLAIM TO FALSIFY: "The R1 milestone (W1 cap=6 + R4 inactivity-end=1.5/rally_thresh=0.20/
min_density=0.30, no W2, no R5) still clears the net-over-seg promotion bar at n=13 ? a
SIGNIFICANT
R2 tolF1 win over baseline AND no honest over-seg regression ? so config-flip PR #204 still
stands."

MY (the orchestrator's) measured n=13 numbers, for you to INDEPENDENTLY confirm or REFUTE ? do
```

```
NOT
trust them, recompute:
  - baseline->milestone tolF1: ?+0.2222 [+0.1522,+0.2859], sign 12/0/1, p=0.0005
  - cap12->cap6 tolF1:         ?+0.1099 [+0.0703,+0.1526], sign 12/0/1, p=0.0005
  - cap6->+R4 tolF1:           ?+0.0397 [+0.0145,+0.0651], sign 9/1/3,  p=0.0215   (ONE loss ?
find it)
  - mean merge_rate baseline->milestone: 0.651 -> 0.161; over_seg_guard NET PASS, no merge_rate
regression
  - new clips: GX010137 tolF1 0.448->0.685, GX010141 0.333->0.431 (both improve)


LENS = the net over-seg guard at n=13. Independently compute per-clip merge_rate and
split_rate at baseline and milestone for all 13. Verify the CATEGORICAL block condition: does
ANY clip's
merge_rate INCREASE baseline->milestone? (If yes, the guard must BLOCK.) Run over_seg_guard and
report
passes / strict_passes / net cost base->cand / no_merge_rate_regress / the merge_rate_regress
list. Confirm
the strict rule BLOCKS purely because of split-count rises (the structural mega->bounded
artifact), not a
real over-merge regression. Also confirm all 13 clips are scored (none dropped by the mixed-
schema corpus).
Adversarially: is there any honest reading under which over-segmentation got WORSE?

Return ONLY the structured verdict. Be precise with numbers.
```

2. assistant (2026-06-18T05:31:47.365Z)

```
I'll verify this claim independently. Let me start by pulling the repo and examining the
structure.
```

3. user (2026-06-18T05:31:48.985Z)

```
f5f339a fix(eval): fusion loaders skip windowing-only files on a mixed golden corpus (#210)
---
analyze_motion.py
clean_state.py
diagnose_rally.py
gemini_video_debugger.py
list_models.py
r1_evidence.py
test_ai_segmenter.py
test_thresholds.py
```

4. user (2026-06-18T05:31:50.192Z)

```
Badminton_BXH_2.rallies.csv
Badminton_BXH_2_golden_features.json
GX010125_highlights.mp4
GX010125_highlights_min5.mp4
GX010125_run
GX010128.rallies.csv
GX010128_golden_features.json
GX010137.rallies.csv
GX010137_golden_features.json
GX010141.rallies.csv
GX010141_golden_features.json
GX030094.rallies.csv
GX030094_golden_features.json
KushagraAviHighlight-Trailor.mp4
MahadevpuraSingles_highlights.mp4
REAL_FOOTAGE_VALIDATION_REPORT.md
ab_experiment_n6.json
adarsh_avi_full_vs_human.csv
```

```
adarsh_avi_singles_golden_features.json
adarsh_avi_vs_human.csv
adarsh_baseline_gated.csv
adarsh_baseline_segments.csv
adarsh_gemini_fullvideo.csv
adarsh_gemini_refined.csv
adarsh_gemini_smoke.csv
adarsh_min5s_segments.csv
adarsh_tuned_segments.csv
badminton_indexer.db
chunks
e2e_reel.mp4
---MANIFEST---
C:/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json
```

5. assistant (2026-06-18T05:31:55.508Z)

Worktree HEAD is f5f339a as expected. Let me write my own verification script.

6. user (2026-06-18T05:31:56.426Z)

```
1       """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3       The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4       golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5       over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6       boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7       number (a single ? misleads; see the design doc).
8
9       Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10      asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11      rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13      Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14      else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15      eligibility graphs that rally intervals produce, so scores are environment-independent.
16      """
17      from __future__ import annotations
18
19      import statistics
20      from typing import Any, Dict, List, Optional, Tuple
21
22      Interval = Tuple[float, float]
23      #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
24      Match = Tuple[int, int, float, float]
25
26      #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
27      #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
28      TAU_START: float = 2.0
29      TAU_END: float = 1.5
30      SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)
31
32
33      def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
```

```
34          return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end
35
36
37      def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
38                             tau_start: float, tau_end: float) -> List[Match]:
39          """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
40          assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
41          graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
42          pairs: List[Tuple[float, float, int, int]] = []
43          for i, p in enumerate(preds):
44              for j, g in enumerate(gts):
45                  if _eligible(p, g, tau_start, tau_end):
46                      # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
47                      pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
48          pairs.sort()
49          used_p: set = set()
50          used_g: set = set()
51          matches: List[Match] = []
52          for _, _, i, j in pairs:
53              if i in used_p or j in used_g:
54                  continue
55              used_p.add(i)
56              used_g.add(j)
57              matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
58          matches.sort(key=lambda mm: mm[1])
59          return matches
60
61
62      def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
63                                tau_start: float, tau_end: float) -> Optional[List[Match]]:
64          """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
65          numpy/scipy are unavailable so the caller falls back to greedy."""
66          try:
67              import numpy as np
68              from scipy.optimize import linear_sum_assignment
69          except Exception:
70              return None
71          n, m = len(preds), len(gts)
72          size = max(n, m)
73          big = 1.0e6
74          cost = np.full((size, size), big, dtype=float)
75          for i, p in enumerate(preds):
76              for j, g in enumerate(gts):
77                  if _eligible(p, g, tau_start, tau_end):
78                      cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
79          rows, cols = linear_sum_assignment(cost)
80          matches: List[Match] = []
81          for i, j in zip(rows.tolist(), cols.tolist()):
82              if i < n and j < m and cost[i, j] < big:  # drop forced ineligible assignments
83                  matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
84          matches.sort(key=lambda mm: mm[1])
85          return matches
86
87
88      def match_1to1(preds: List[Interval], gts: List[Interval],
89                     tau_start: float = TAU_START, tau_end: float = TAU_END
90                     ) -> Tuple[List[Match], float, float, float]:
91          """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
92          (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
```

```
scipy is
93          present; deterministic greedy otherwise."""
94          n, m = len(preds), len(gts)
95          if n == 0 or m == 0:
96              return [], 0.0, 0.0, 0.0
97          matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
98          if matches is None:
99              matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
100         k = len(matches)
101         precision = k / n
102         recall = k / m
103         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
0.0
104         return matches, precision, recall, f1
105
106
107     def _percentile(xs: List[float], p: float) -> float:
108         """Linear-interpolated percentile (pure stdlib)."""
109         if not xs:
110             return 0.0
111         s = sorted(xs)
112         k = (len(s) - 1) * p / 100.0
113         lo = int(k)
114         hi = min(lo + 1, len(s) - 1)
115         return s[lo] + (s[hi] - s[lo]) * (k - lo)
116
117
118     def _overlap(a: Interval, b: Interval) -> float:
119         return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
120
121
122     def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
123         """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
124         end open" diagnostic that aims R4.
125
126         Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
127         positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
128         ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
129         this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
130         to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
131         def dist(vals: List[float]) -> Dict[str, float]:
132             av = [abs(v) for v in vals]
133             return {
134                 "signed_median": statistics.median(vals) if vals else 0.0,
135                 "abs_median": statistics.median(av) if av else 0.0,
136                 "abs_p25": _percentile(av, 25.0),
137                 "abs_p75": _percentile(av, 75.0),
138                 "n": float(len(vals)),
139             }
140         ds: List[float] = []
141         de: List[float] = []
142         for g in gts:
143             best = max(preds, key=lambda p: _overlap(p, g), default=None)
144             if best is not None and _overlap(best, g) > 0.0:
145                 ds.append(best[0] - g[0])
146                 de.append(best[1] - g[1])
147         return {"dstart": dist(ds), "dend": dist(de)}
148
```

```
149
150     def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
151         """The over-segmentation GUARD (per-video no-regression in promotion):
``split_count`` (?2
152         preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
bipartite
153         overlap-graph in ``segmentation_metrics.merge_split_report``."""
154         from backend.eval.segmentation_metrics import merge_split_report
155         r = merge_split_report(preds, gts)
156         return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
157
158
159     # ------------------------------------------------------------------------ #
160     # R1 ? net over-segmentation promotion guard
161     # ------------------------------------------------------------------------ #
162     #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
163     #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
164     #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
165     #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
166     #: the asymmetry is the principled default, not a number tuned to force a verdict.)
167     WEIGHT_MERGE: float = 2.0
168     WEIGHT_SPLIT: float = 1.0
169
170
171     def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
172                        weight_merge: float = WEIGHT_MERGE, weight_split: float =
WEIGHT_SPLIT,
173                        use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
174         """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).
175
176         ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
177         emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
178         ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
179         position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.
180
181         **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
``cand`` raises
182         ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
183         windowing structure, an increase = a real regression) but mis-fires on a
*structural* change
184         (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
185         ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
186         ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
187         that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
rallies hidden).
188         So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
189         over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
190         ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)
191
```

```
192         **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
193         require BOTH:
194           1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
195           2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
196              of a video's rallies is the one over-merge regression that is never acceptable,
the honest
197              "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
198         With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction
of GT rallies
199         affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.
200
201         **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
202         strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
203         """
204         def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
205             if base and cand and all("name" in b for b in base) and all("name" in c for c in
cand):
206                 cmap = {c["name"]: c for c in cand}
207                 return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
208             n = min(len(base), len(cand))
209             return [(base[i], cand[i]) for i in range(n)]
210
211         def _cost(row: Dict[str, Any]) -> float:
212             if use_rates:
213                 return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
214             return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])
215
216         pairs = _aligned()
217         n = len(pairs)
218         base_costs = [_cost(b) for b, _ in pairs]
219         cand_costs = [_cost(c) for _, c in pairs]
220         base_net = (sum(base_costs) / n) if n else 0.0
221         cand_net = (sum(cand_costs) / n) if n else 0.0
222         net_delta = cand_net - base_net
223
224         # Per-video regressions, reported for transparency.
225         new_merges: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
226         new_splits: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
227         merge_rate_regress: List[Dict[str, Any]] = []    # RATE increases (the real over-
merge regression)
228         for b, c in pairs:
229             if float(c["merge_count"]) > float(b["merge_count"]) + eps:
230                 new_merges.append({"name": b.get("name", ""),
231                                    "base": float(b["merge_count"]), "cand":
float(c["merge_count"])})
232             if float(c["split_count"]) > float(b["split_count"]) + eps:
233                 new_splits.append({"name": b.get("name", ""),
234                                    "base": float(b["split_count"]), "cand":
float(c["split_count"])})
235             if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
236                 merge_rate_regress.append({"name": b.get("name", ""),
237                                            "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])})
238
239         strict_passes = (not new_merges) and (not new_splits)
```

```
240         net_cost_ok = net_delta <= eps
241         no_merge_rate_regress = not merge_rate_regress
242         passes = bool(net_cost_ok and no_merge_rate_regress)
243         return {
244             "passes": passes,
245             "strict_passes": strict_passes,
246             "net_cost_ok": net_cost_ok,
247             "no_merge_rate_regress": no_merge_rate_regress,
248             "base_net": base_net,
249             "cand_net": cand_net,
250             "net_delta": net_delta,
251             "n": n,
252             "use_rates": use_rates,
253             "weight_merge": weight_merge,
254             "weight_split": weight_split,
255             "new_merges": new_merges,
256             "new_splits": new_splits,
257             "merge_rate_regress": merge_rate_regress,
258         }
259
260
261     def tolerance_report(preds: List[Interval], gts: List[Interval],
262                          tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
263         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
264         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
265         return {
266             "tau_start": tau_start, "tau_end": tau_end,
267             "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
268             "precision": precision, "recall": recall, "f1": f1,
269             "boundary_error": boundary_error_report(preds, gts),
270             "over_seg": over_seg_report(preds, gts),
271         }
272
273
274     def tolerance_sweep(preds: List[Interval], gts: List[Interval],
275                         taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
276         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
277         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
278
```

7. user (2026-06-18T05:31:58.096Z)

```
__init__.py
__pycache__
ablation.py
annotations.py
audio
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
golden_fixtures.py
```

```
gt_loader.py
harness.py
metrics.py
per_user_eval.py
per_user_gate.py
per_user_regression.py
promotion.py
rally_seg_eval.py
rally_seq_proto.py
regression.py
score_calibration.py
segmentation_metrics.py
serve_contrast.py
served_gate_a.py
splits.py
tolerance_metrics.py
training.py
windowing.py
---SERVED---
6:deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED pipeline
14:SERVED preset is **single-sourced from the same Pydantic config defaults production reads**
20:The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
38:    "SERVED",
103:    """The SERVED windowing, lifted from the SAME Pydantic config defaults production reads.
```

8. assistant (2026-06-18T05:32:00.451Z)

Now let me look at the `rally_seg_eval` and `eval_stats` public APIs to understand the row schema and `evaluate`/`paired_delta_ci` signatures.

9. user (2026-06-18T05:32:03.925Z)

```
52:    tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs precision) +
the
61:    tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
69:        "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
74:        "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
81:_AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate", "segment_count_ratio",
83:             "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median", "dend_abs_median")
101:def load_golden(manifest_path: str = DEFAULT_MANIFEST,
114:        out.append({"name": v["name"], "trajectory": v["trajectory"],
147:def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
158:        row["name"] = v["name"]
172:    sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also reports ?
on the
179:    by_name_b = {r["name"]: r for r in b["rows"]}
180:    paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
182:    for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate", "segment_count_ratio"):
208:             f"no_merge_rate_regress={g['no_merge_rate_regress']})",
212:    if g["merge_rate_regress"]:
214:                           for r in g["merge_rate_regress"])
215:        lines.append(f"  ? merge_rate REGRESSED on: {names}")
233:    lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
237:                     f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
242:              f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}"]
245:    if rows and "tol_f1" in rows[0]:
251:            lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f} | "
254:        lines += ["", f"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "
```

10. user (2026-06-18T05:32:04.756Z)

```
32:def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
57:def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
69:def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
90:def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
112:def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
132:def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
List[Optional[Any]]:
```

11. assistant (2026-06-18T05:32:10.192Z)

Let me check the row schema keys and the `paired_delta_ci` return shape, plus note the row key is `tol_f1` (not `tol_f1` aliased as `tolF1`).

12. user (2026-06-18T05:32:12.553Z)

```
45
46          Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
47          over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
48          merged mega-window is *visible* even when IoU-F1 isn't moved.
49
50          Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
51          tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
52          tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
53          start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
54          """
55          from backend.eval import tolerance_metrics as tm
56          ts = tm.TAU_START if tau_start is None else tau_start
57          te = tm.TAU_END if tau_end is None else tau_end
58          sc = metrics.score(preds, gts, iou_threshold=iou)
59          f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
60          ms = segmentation_metrics.merge_split_report(preds, gts)
61          tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
62          be = tm.boundary_error_report(preds, gts)  # unconditional best-overlap (the
R4-aiming diagnostic)
63          osr = tm.over_seg_report(preds, gts)
64          return {
65              "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
66              "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
67              "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
68              "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
69              "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
70              "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
71              "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
72              # --- R2 (tolerance-match) ? the task-faithful headline block ---
73              "tau_start": ts, "tau_end": te,
74              "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
75              "tol_matched": float(len(tol_matches)),
76              "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
77              "split_count": osr["split_count"], "merge_count": osr["merge_count"],
78          }
79
80
81      _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
82                   "precision", "recall",
83                   "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
84
```

```
85
86      def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
87          """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
88
89          Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
90          leave-one-video-out discipline (windows within a video are correlated)."""
91          out: Dict[str, Any] = {"n": len(rows)}
92          for k in _AGG_KEYS:
93              vals = [r[k] for r in rows]
94              out[k] = eval_stats.mean_with_ci(vals) if vals else None
95          return out
96
97
98      # ---------------------------------------------------------------------- #
99      # Golden corpus I/O
100     # ---------------------------------------------------------------------- #
101     def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102                     features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
104         intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
105         with open(manifest_path) as fh:
106             manifest = json.load(fh)
107         out: List[Dict[str, Any]] = []
108         for v in manifest:
109             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
110             gts: List[Interval] = []
111             if os.path.exists(feat):
112                 with open(feat) as fh:
113                     gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
114             out.append({"name": v["name"], "trajectory": v["trajectory"],
115                         "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
"gts": gts})
116         return out
117
118
119     def windows_for(video: Dict[str, Any], preset: WindowingPreset,
120                     min_crossings: int = 0) -> List[Interval]:
121         """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
122
123         ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
124         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
125         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
126         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
127         windowing (W1) leaves behind.
128
129         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
130         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
131         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
132         """
133         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
134         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
135         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
136         ivs = windowing.windows_to_intervals(wins)
```

```
137         if min_crossings > 0 and ivs:
138             from backend.pipeline.detectors.rally_gate import gate_windows
139             gated = [(g.start, g.end)
140                      for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
141                      if g.kept]
142             # Gating emptied a non-empty video ? keep the ungated windows.
143             ivs = gated if gated else ivs
144         return ivs
145
146
147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                  iou: float = DEFAULT_IOU, min_crossings: int = 0,
149                  tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)
157             row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161                 "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
166                 cand_min_crossings: int = 0,
167                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168                 guard_weight_merge: Optional[float] = None,
169                 guard_weight_split: Optional[float] = None,
170                 guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
172         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
```

13. user (2026-06-18T05:32:13.252Z)

```
69      def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70          """Two-sided exact sign test over per-fold deltas (B ? A).
71
72          ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73          excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74          distribution-free "did B win on enough *videos*?" check.
75          """
76          wins = sum(1 for d in deltas if d > eps)
77          losses = sum(1 for d in deltas if d < -eps)
78          ties = len(deltas) - wins - losses
79          n_eff = wins + losses
80          if n_eff == 0:
81              p = 1.0
82          else:
83              k = min(wins, losses)
84              tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
```

```
85              p = min(1.0, 2.0 * tail)
86          return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87                  "n_effective": float(n_eff), "p_value": p}
88
89
90      def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                          confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                          eps: float = 0.0) -> Dict[str, Any]:
93          """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95          ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
by
96          fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
sign
97          test agrees ? not when a single mean ticked up.
98          """
99          n = min(len(a_values), len(b_values))
100         deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
101         summary = mean_with_ci(deltas, confidence, n_boot, seed)
102         return {
103             "mean_delta": summary["mean"],
104             "ci_low": summary["ci_low"],
105             "ci_high": summary["ci_high"],
106             "n": float(n),
107             "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
108             "sign_test": paired_sign_test(deltas, eps),
109         }
110
111
112     def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
113         """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
unit.
```

14. assistant (2026-06-18T05:32:15.380Z)

I have the full picture of the API. Now let me write my own verification script.

15. assistant (2026-06-18T05:32:35.906Z)

```
import os, json
from dataclasses import replace
from backend.eval import rally_seg_eval as rse, eval_stats
from backend.eval.windowing import SERVED
from backend.eval.tolerance_metrics import over_seg_guard

OUT = "C:/Users/avidu/Projects/badminton-highlight-indexer/output"

golden = [g for g in rse.load_golden(os.path.join(OUT, "human_lovo_manifest.json"), OUT)
          if g["gts"] and os.path.exists(g["trajectory"])]
print("N golden (gts + trajectory exists):", len(golden))
print("names:", sorted(g["name"] for g in golden))

baseline  = SERVED
cap12     = replace(SERVED, max_window_duration=12.0)
cap6      = replace(SERVED, max_window_duration=6.0)
milestone = replace(SERVED, max_window_duration=6.0, inactivity_end=1.5,
                    inactivity_rally_thresh=0.20, inactivity_min_density=0.30)

def rows(p):
    return {r["name"]: r for r in rse.evaluate(golden, p)["rows"]}

b_rows = rows(baseline)
m_rows = rows(milestone)
c12_rows = rows(cap12)
c6_rows = rows(cap6)
```

```
names = sorted(b_rows.keys())
print("\nN scored rows: baseline=%d milestone=%d cap12=%d cap6=%d" %
      (len(b_rows), len(m_rows), len(c12_rows), len(c6_rows)))
print("all names scored in BOTH baseline & milestone:", set(b_rows) == set(m_rows) ==
set(names))
```

---- Per-clip merge_rate / split_rate baseline -> milestone ----

```
print("\n%-12s | %7s %7s | %7s %7s | %8s %8s | %8s %8s" %
      ("clip", "b_mr", "m_mr", "b_sr", "m_sr", "b_mcnt", "m_mcnt", "b_scnt", "m_scnt"))
any_mr_increase = False
for n in names:
    b, m = b_rows[n], m_rows[n]
    flag = ""
    if m["merge_rate"] > b["merge_rate"] + 1e-9:
        any_mr_increase = True
        flag = "  <-- merge_rate UP"
    print("%-12s | %7.3f %7.3f | %7.3f %7.3f | %8d %8d | %8d %8d%s" %
          (n, b["merge_rate"], m["merge_rate"], b["split_rate"], m["split_rate"],
           int(b["merge_count"]), int(m["merge_count"]),
           int(b["split_count"]), int(m["split_count"]), flag))

import statistics
print("\nmean merge_rate baseline:  %.4f" % statistics.mean(b_rows[n]["merge_rate"] for n in
names))
print("mean merge_rate milestone: %.4f" % statistics.mean(m_rows[n]["merge_rate"] for n in
names))
print("mean split_rate baseline:  %.4f" % statistics.mean(b_rows[n]["split_rate"] for n in
names))
print("mean split_rate milestone: %.4f" % statistics.mean(m_rows[n]["split_rate"] for n in
names))
print("ANY clip merge_rate INCREASE baseline->milestone:", any_mr_increase)
```

---- Paired tolF1 sign-tests ----

```
def delta(a_rows, b_rows_, key):
    a = [a_rows[n][key] for n in names]
    b = [b_rows_[n][key] for n in names]
    return eval_stats.paired_delta_ci(a, b)

print("\n=== tolF1 paired deltas ===")
for label, (ar, br) in [("baseline->milestone", (b_rows, m_rows)),
                        ("cap12->cap6", (c12_rows, c6_rows)),
                        ("cap6->milestone(+R4)", (c6_rows, m_rows))]:
    d = delta(ar, br, "tol_f1")
    st = d["sign_test"]
    print("%-22s  ?%+.4f [%+.4f,%+.4f]  sign %d/%d/%d p=%.4f  ci_excl0=%s" %
          (label, d["mean_delta"], d["ci_low"], d["ci_high"],
           int(st["wins"]), int(st["losses"]), int(st["ties"]), st["p_value"],
           d["ci_excludes_zero"]))
```

find the ONE loss in cap6->milestone

```
print("\ncap6->milestone per-clip tolF1 (find the loss):")
for n in names:
    d = m_rows[n]["tol_f1"] - c6_rows[n]["tol_f1"]
    tag = "  <== LOSS" if d < -1e-9 else ("  (tie)" if abs(d) <= 1e-9 else "")
    print("  %-12s c6=%.4f mile=%.4f  ?%+.4f%s" % (n, c6_rows[n]["tol_f1"], m_rows[n]["tol_f1"],
d, tag))
```

---- over_seg_guard ----

```
print("\n=== over_seg_guard(base=baseline, cand=milestone) ===")
g = over_seg_guard(list(b_rows.values()), list(m_rows.values()))
for k in ("passes", "strict_passes", "net_cost_ok", "no_merge_rate_regress",
          "base_net", "cand_net", "net_delta", "n"):
```

```
    print("  %-22s %s" % (k, g[k]))
print("  new_merges (count up):", g["new_merges"])
print("  new_splits (count up):", [(x["name"], int(x["base"]), int(x["cand"])) for x in
g["new_splits"]])
print("  merge_rate_regress:    ", g["merge_rate_regress"])
```

new clips check

```
print("\nnew-clip tolF1 baseline->milestone:")
for n in ("GX010137", "GX010141"):
    if n in b_rows:
        print("  %-12s %.4f -> %.4f" % (n, b_rows[n]["tol_f1"], m_rows[n]["tol_f1"]))
```

16. user (2026-06-18T05:32:35.921Z)

```
File created successfully at: C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/verify-n13/scratch/lens_n13_overseg.py (file state is current in your
context ? no need to Read it back)
```

17. user (2026-06-18T05:32:47.828Z)

```
N golden (gts + trajectory exists): 13
names: ['Badminton_BXH_2', 'GX010128', 'GX010137', 'GX010141', 'GX030094', 'adarsh_avi_singles',
'gbaaddy', 'kushagra_singles', 'largetest_doubles', 'mahadevpura_1', 'mahadevpura_2',
'mahadevpura_singles', 'testlarge_short']

N scored rows: baseline=13 milestone=13 cap12=13 cap6=13
all names scored in BOTH baseline & milestone: True

clip         |    b_mr    m_mr |    b_sr    m_sr |   b_mcnt   m_mcnt |   b_scnt   m_scnt
Badminton_BXH_2 |   1.000   0.091 |   0.000   0.091 |        3        2 |        0        4
GX010128     |   0.962   0.000 |   0.000   0.038 |       12        0 |        0        2
GX010137     |   0.176   0.000 |   0.000   0.029 |        3        0 |        0        1
GX010141     |   0.655   0.448 |   0.000   0.000 |        5        6 |        0        0
GX030094     |   0.561   0.000 |   0.000   0.146 |        8        0 |        0        6
adarsh_avi_singles |   0.104   0.000 |   0.000   0.135 |        5        0 |        0       13
gbaaddy      |   0.854   0.042 |   0.000   0.167 |        9        1 |        0        8
kushagra_singles |   1.000   0.188 |   0.000   0.188 |        2        3 |        0        6
largetest_doubles |   0.163   0.000 |   0.000   0.102 |        4        0 |        0        5
mahadevpura_1 |   1.000   0.800 |   0.000   0.000 |        1        2 |        0        0
mahadevpura_2 |   0.975   0.525 |   0.000   0.100 |        3        9 |        0        4
mahadevpura_singles |   0.516   0.000 |   0.000   0.226 |        6        0 |        0        7
testlarge_short |   0.500   0.000 |   0.000   0.167 |        1        0 |        0        1

mean merge_rate baseline:  0.6513
mean merge_rate milestone: 0.1610
mean split_rate baseline:  0.0000
mean split_rate milestone: 0.1069
ANY clip merge_rate INCREASE baseline->milestone: False

=== tolF1 paired deltas ===
baseline->milestone     ?+0.2222 [+0.1558,+0.2894]  sign 12/0/1 p=0.0005  ci_excl0=True
cap12->cap6             ?+0.1099 [+0.0700,+0.1522]  sign 12/0/1 p=0.0005  ci_excl0=True
cap6->milestone(+R4)    ?+0.0397 [+0.0155,+0.0652]  sign 9/1/3 p=0.0215  ci_excl0=True

cap6->milestone per-clip tolF1 (find the loss):
  Badminton_BXH_2 c6=0.2273 mile=0.3279  ?+0.1006
  GX010128     c6=0.3472 mile=0.4662  ?+0.1189
  GX010137     c6=0.7273 mile=0.6849  ?-0.0423  <== LOSS
  GX010141     c6=0.3929 mile=0.4314  ?+0.0385
  GX030094     c6=0.3538 mile=0.4194  ?+0.0655
  adarsh_avi_singles c6=0.4957 mile=0.5520  ?+0.0564
  gbaaddy      c6=0.1854 mile=0.2778  ?+0.0923
  kushagra_singles c6=0.0741 mile=0.0779  ?+0.0038
  largetest_doubles c6=0.5645 mile=0.5833  ?+0.0188
```

```
  mahadevpura_1 c6=0.0000 mile=0.0000  ?+0.0000  (tie)
  mahadevpura_2 c6=0.1220 mile=0.1220  ?+0.0000  (tie)
  mahadevpura_singles c6=0.4667 mile=0.5301  ?+0.0635
  testlarge_short c6=0.2000 mile=0.2000  ?+0.0000  (tie)

=== over_seg_guard(base=baseline, cand=milestone) ===
  passes                True
  strict_passes         False
  net_cost_ok           True
  no_merge_rate_regress  True
  base_net              1.3025976530825853
  cand_net              0.4289172588246174
  net_delta             -0.873680394257968
  n                     13
  new_merges (count up): [{'name': 'kushagra_singles', 'base': 2.0, 'cand': 3.0}, {'name':
'mahadevpura_2', 'base': 3.0, 'cand': 9.0}, {'name': 'mahadevpura_1', 'base': 1.0, 'cand': 2.0},
{'name': 'GX010141', 'base': 5.0, 'cand': 6.0}]
  new_splits (count up): [('adarsh_avi_singles', 0, 13), ('kushagra_singles', 0, 6),
('largetest_doubles', 0, 5), ('gbaaddy', 0, 8), ('testlarge_short', 0, 1),
('mahadevpura_singles', 0, 7), ('mahadevpura_2', 0, 4), ('GX010128', 0, 2), ('GX030094', 0, 6),
('Badminton_BXH_2', 0, 4), ('GX010137', 0, 1)]
  merge_rate_regress:     []

new-clip tolF1 baseline->milestone:
  GX010137     0.4478 -> 0.6849
  GX010141     0.3333 -> 0.4314
```

18. assistant (2026-06-18T05:32:51.811Z)

All numbers computed. Let me examine the one nuance the orchestrator flagged: `strict_passes=False`. The orchestrator's claim says it should block "purely because of split-count rises." But my output shows `new_merges` is NON-empty ? there are merge_COUNT increases on 4 clips. Let me verify the strict rule's exact failure attribution and confirm those merge_count rises do NOT correspond to merge_RATE rises (i.e., no honest over-merge regression).

19. user (2026-06-18T05:33:04.104Z)

```
Clips with merge_COUNT up (what strict trips on) -- check their merge_RATE:
  GX010141           mcnt 5->6  mrate 0.655->0.448  (rate down? True)
  kushagra_singles   mcnt 2->3  mrate 1.000->0.188  (rate down? True)
  mahadevpura_1      mcnt 1->2  mrate 1.000->0.800  (rate down? True)
  mahadevpura_2      mcnt 3->9  mrate 0.975->0.525  (rate down? True)

Clips with split_COUNT up: 11 of 13
Baseline split_count all zero? True
Baseline merge_rate==1.0 (mega-window swallow) clips: ['kushagra_singles', 'mahadevpura_1',
'Badminton_BXH_2']
```

20. assistant (2026-06-18T05:33:11.038Z)

This confirms the structural-artifact mechanism precisely. The 4 clips with merge_COUNT rises ALL have merge_RATE falling (the mega-window?bounded artifact: one window swallowing N rallies becomes several windows each gluing 2 neighbors ? higher count, lower fraction-hidden). Strict trips on both `new_merges` (count) and `new_splits` (count), but neither reflects a real over-merge regression because merge_rate falls everywhere.

One refinement to the orchestrator's lens framing: strict blocks due to BOTH split-count AND merge-count rises (not "purely" split-count). But the substantive point holds ? no merge_RATE regression. Let me also adversarially probe whether any honest reading makes over-segmentation worse, by examining the split-rate rise on `adarsh_avi_singles` (0?13 splits) which is the largest.

21. user (2026-06-18T05:33:26.443Z)

```
NET cost (merge-weighted 2:1):  base 1.3026 -> cand 0.4289  (down? True)
NET cost (EQUAL weights 1:1):   base 0.6513 -> cand 0.2679  (down? True)
NET cost (split-HEAVY 1:2):     base 0.6513 -> cand 0.3748  (down? True)
NET cost (split-PUNITIVE 1:5):  base 0.6513 -> cand 0.6953  (down? False)

clips with BOTH merge_rate AND split_rate up: []
mean num_pred base 23.77 -> cand 57.69
```

22. assistant (2026-06-18T05:33:32.258Z)

The adversarial probe is thorough and revealing. Net cost falls under the default (2:1), equal (1:1), and even split-heavy (1:2) weights. It only rises under a split-PUNITIVE 1:5 weighting ? which would value a fragmented-but-found rally as 2.5× worse than a hidden rally, an indefensible inversion of the recall-meaningful priority (a merge hides a rally; a split still finds it). No clip regresses on BOTH axes. The split-rate rise (0?0.107) and pred-count inflation (23.8?57.7) are the structural mega?bounded artifact, not new over-merging.

I have everything needed. Let me record the verified verdict.

23. user (2026-06-18T05:33:54.718Z)

Structured output provided successfully

24. assistant (2026-06-18T05:34:02.974Z)

Verdict recorded: CONFIRMS. The R1 milestone clears the net-over-seg promotion bar at n=13. All 13 clips scored (none dropped by the mixed-schema corpus), no clip's merge_rate increases baseline?milestone, `over_seg_guard` returns `passes=True` / `merge_rate_regress=[]` / `net_delta=?0.8737`, and the tolF1 wins reproduce within bootstrap jitter. The one substantive correction to the lens framing: `strict_passes=False` is driven by count rises on BOTH split AND merge axes (4 merge-count rises), but every one of those merge-count rises has a FALLING merge_rate ? the mega?bounded structural artifact, not a real over-merge regression. Over-segmentation only reads as "worse" under an indefensible split-punitive 1:5 weighting.